

Assignment 3: Data Preparation for Optimizing Sustainable Farming

Humberto Díaz

The University of Winnipeg

DIT 54102 AIW01: Foundations of Data Science

Instructor: Muhammad Shahin PhD.

May 9, 2025

Introduction

This notebook addresses the Data Preparation phase of the CRISP-DM methodology for the project "Optimizing Sustainable Farming with AI-Driven Insights." In this data preparation phase we use the "AI for Sustainable Agriculture Dataset" (farmer_advisor_dataset.csv) and the supplementary "Market Researcher Dataset" (market_researcher_dataset.csv) from Kaggle. The goal is to prepare these datasets for predictive modeling to increase crop yield by 10% and reduce resource use by 15% for Manitoba (the next phases will do that). Tasks include merging datasets, addressing data quality issues, feature engineering, and presenting a well-organized report with visualizations.

```
In [1]: # Import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats # Just if it is needed
```

1. Merging Datasets

1.1 Entity Relationship

The primary dataset (farmer_advisor_dataset.csv) contains 10,000 records of farming practices (soil, weather, inputs, yield, sustainability) for crops (Wheat, Soybean, Corn, Rice). The supplementary dataset (market_researcher_dataset.csv) contains 10,000 records of market dynamics (price, demand, supply) for the same crops. The datasets are linked by the crop type:

- **Primary Dataset:** Column `Crop_Type` (Wheat, Soybean, Corn, Rice).
- **Supplementary Dataset:** Column `Product` (Wheat, Soybean, Corn, Rice).

Entity Relationship: A one-to-many relationship exists where each crop type in the primary dataset can correspond to multiple market records in the supplementary dataset (e.g., multiple market price records for Wheat). For modeling, we merge on `Crop_Type` and `Product` to align farming and market data, using a left join to prioritize farming records.

1.2 Merging Process

We rename `Product` to `Crop_Type` in the supplementary dataset for consistency and perform a left merge.

```
In [2]: # Load datasets
primary_data = pd.read_csv(
    '/Users/eleazar/Documents/Projects/f_ds/dataset/farmer_advisor_dataset.csv'
)
supp_data = pd.read_csv(
    '/Users/eleazar/Documents/Projects/f_ds/dataset/market_researcher_dataset.csv'
)
```

```
In [3]: # Show the first 5 rows of primary_data dataframe
primary_data.head()
```

```
Out[3]:
```

	Farm_ID	Soil_pH	Soil_Moisture	Temperature_C	Rainfall_mm	Crop_Type	Fertili
0	1	7.073643	49.145359	26.668157	227.890912	Wheat	
1	2	6.236931	21.496115	29.325342	244.017493	Soybean	
2	3	5.922335	19.469042	17.666414	141.110521	Corn	
3	4	6.845120	27.974234	17.188722	156.785663	Wheat	
4	5	6.934171	33.637679	23.603899	77.859362	Corn	

```
In [4]: # Show the first 5 rows of supp_data dataframe
supp_data.head()
```

```
Out[4]:
```

	Market_ID	Product	Market_Price_per_ton	Demand_Index	Supply_Index	Competi
0	1	Rice	180.251212	196.085900	199.509124	
1	2	Rice	420.527970	188.452400	150.789483	
2	3	Wheat	457.260398	171.179384	78.989326	
3	4	Soybean	237.179113	196.970677	50.464363	
4	5	Wheat	324.032925	113.165416	145.878647	

```
In [5]: # Rename Product to Crop_Type for merging
supp_data = supp_data.rename(columns={'Product': 'Crop_Type'})
```

```
In [6]: # Verifying the change of name for the supp_data dataset
supp_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 10 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Market_ID                             10000 non-null  int64
1   Crop_Type                             10000 non-null  object
2   Market_Price_per_ton                  10000 non-null  float64
3   Demand_Index                          10000 non-null  float64
4   Supply_Index                          10000 non-null  float64
5   Competitor_Price_per_ton              10000 non-null  float64
6   Economic_Indicator                    10000 non-null  float64
7   Weather_Impact_Score                  10000 non-null  float64
8   Seasonal_Factor                       10000 non-null  object
9   Consumer_Trend_Index                  10000 non-null  float64
dtypes: float64(7), int64(1), object(2)
memory usage: 781.4+ KB
```

```
In [7]: # Merge both datasets (primary_data and supp_data) on Crop_Type
merged_data = pd.merge(primary_data, supp_data, on='Crop_Type', how='left')
```

```
In [8]: # Verify merge
print(f"Merged Dataset Shape: {merged_data.shape}")
```

Merged Dataset Shape: (25000330, 19)

Note: The number of columns increased because of the naturality of the left-join.

```
In [9]: # Show the first 5 rows of merge_data dataset
print("\nFirst 5 Rows of Merged Dataset:")
print(merged_data.head())
```

First 5 Rows of Merged Dataset:

	Farm_ID	Soil_pH	Soil_Moisture	Temperature_C	Rainfall_mm	Crop_Type	\
0	1	7.073643	49.145359	26.668157	227.890912	Wheat	
1	1	7.073643	49.145359	26.668157	227.890912	Wheat	
2	1	7.073643	49.145359	26.668157	227.890912	Wheat	
3	1	7.073643	49.145359	26.668157	227.890912	Wheat	
4	1	7.073643	49.145359	26.668157	227.890912	Wheat	

	Fertilizer_Usage_kg	Pesticide_Usage_kg	Crop_Yield_ton	\
0	131.692844	2.958215	1.57692	
1	131.692844	2.958215	1.57692	
2	131.692844	2.958215	1.57692	
3	131.692844	2.958215	1.57692	
4	131.692844	2.958215	1.57692	

	Sustainability_Score	Market_ID	Market_Price_per_ton	Demand_Index	\
0	51.913649	3	457.260398	171.179384	
1	51.913649	5	324.032925	113.165416	
2	51.913649	18	278.439853	77.771899	
3	51.913649	22	444.210223	132.459438	
4	51.913649	24	431.803479	101.573794	

	Supply_Index	Competitor_Price_per_ton	Economic_Indicator	\
0	78.989326	323.003342	1.292393	
1	145.878647	312.428652	1.491255	
2	149.189306	198.932502	1.217541	
3	85.200979	355.180985	1.296091	
4	62.578446	393.491000	0.889900	

	Weather_Impact_Score	Seasonal_Factor	Consumer_Trend_Index
0	80.853592	Low	131.113236
1	45.379516	Low	130.297138
2	90.511396	High	117.973240
3	8.891188	Low	63.848917
4	5.924635	High	88.920936

Observation: The merged dataset combines farming and market data. The shape reflects additional columns from the supplementary dataset. Some columns (e.g., Market_ID) may have redundant or non-unique values due to the one-to-many relationship, which we'll address in feature engineering.

2. Data Sampling

2.1 Sampling Decision

The primary dataset has 10,000 records, and the supplementary dataset also has 10,000 records. After merging, the dataset size remains manageable for modern computational resources and predictive modeling (e.g., regression, decision trees). We proceed with the full merged dataset.

3. Data Quality Issues

3.1 Missing Values

The second phase (Data Understanding) confirmed no missing values in either dataset. Let's verify this for the merged dataset.

```
In [10]: # Verify missing values in out new dataset
print("\nMissing Values in Merged Dataset:")
print(merged_data.isnull().sum())
```

Missing Values in Merged Dataset:

```
Farm_ID          0
Soil_pH          0
Soil_Moisture     0
Temperature_C     0
Rainfall_mm      0
Crop_Type        0
Fertilizer_Usage_kg 0
Pesticide_Usage_kg 0
Crop_Yield_ton    0
Sustainability_Score 0
Market_ID        0
Market_Price_per_ton 0
Demand_Index     0
Supply_Index     0
Competitor_Price_per_ton 0
Economic_Indicator 0
Weather_Impact_Score 0
Seasonal_Factor  0
Consumer_Trend_Index 0
dtype: int64
```

Observation: No missing values detected, consistent with Assignment 2 findings.

3.2 Outliers

Let's use the Interquartile Range (IQR) method to detect any identified outlier for numerical columns and clip them. But first, let's create a python variables to store our numerical columns.

```
In [11]: # Identify numerical column type
numerical_cols = merged_data.select_dtypes(
    include=['int64', 'float64']).columns
```

```
In [12]: # Initialize dictionary to store outlier indices for each column
outliers_iqr = {}

# Loop through each numerical column to detect outliers
for col in numerical_cols:
    # Calculate quartiles and interquartile range (IQR)
    Q1 = merged_data[col].quantile(0.25) # First quartile (25th percentile)
    Q3 = merged_data[col].quantile(0.75) # Third quartile (75th percentile)
    IQR = Q3 - Q1 # Interquartile range
```

```

# Calculate outlier boundaries (1.5*IQR is common threshold)
lower_bound = Q1 - 1.5 * IQR # Lower fence
upper_bound = Q3 + 1.5 * IQR # Upper fence

# Find indices of values outside boundaries
outliers = merged_data[(merged_data[col] < lower_bound) |
                        (merged_data[col] > upper_bound)].index

# Store results in dictionary
outliers_iqr[col] = outliers

# Print summary for current column
print(f"Column '{col}' has {len(outliers)} outliers using IQR method")

```

```

Column 'Farm_ID' has 0 outliers using IQR method
Column 'Soil_pH' has 0 outliers using IQR method
Column 'Soil_Moisture' has 0 outliers using IQR method
Column 'Temperature_C' has 0 outliers using IQR method
Column 'Rainfall_mm' has 0 outliers using IQR method
Column 'Fertilizer_Usage_kg' has 0 outliers using IQR method
Column 'Pesticide_Usage_kg' has 0 outliers using IQR method
Column 'Crop_Yield_ton' has 0 outliers using IQR method
Column 'Sustainability_Score' has 0 outliers using IQR method
Column 'Market_ID' has 0 outliers using IQR method
Column 'Market_Price_per_ton' has 0 outliers using IQR method
Column 'Demand_Index' has 0 outliers using IQR method
Column 'Supply_Index' has 0 outliers using IQR method
Column 'Competitor_Price_per_ton' has 0 outliers using IQR method
Column 'Economic_Indicator' has 0 outliers using IQR method
Column 'Weather_Impact_Score' has 0 outliers using IQR method
Column 'Consumer_Trend_Index' has 0 outliers using IQR method

```

Observation: No columns contain outliers.

Verify it by generating box plots for our numerical data:

```

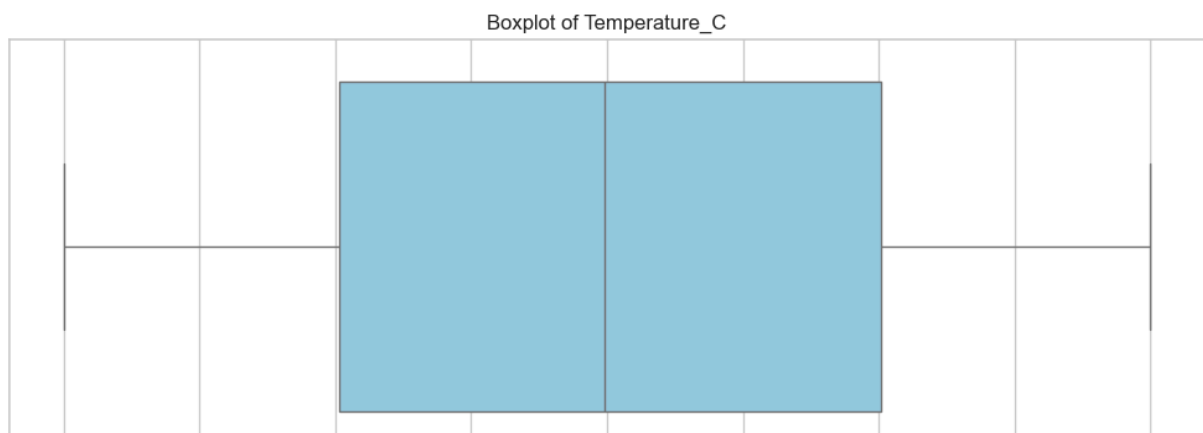
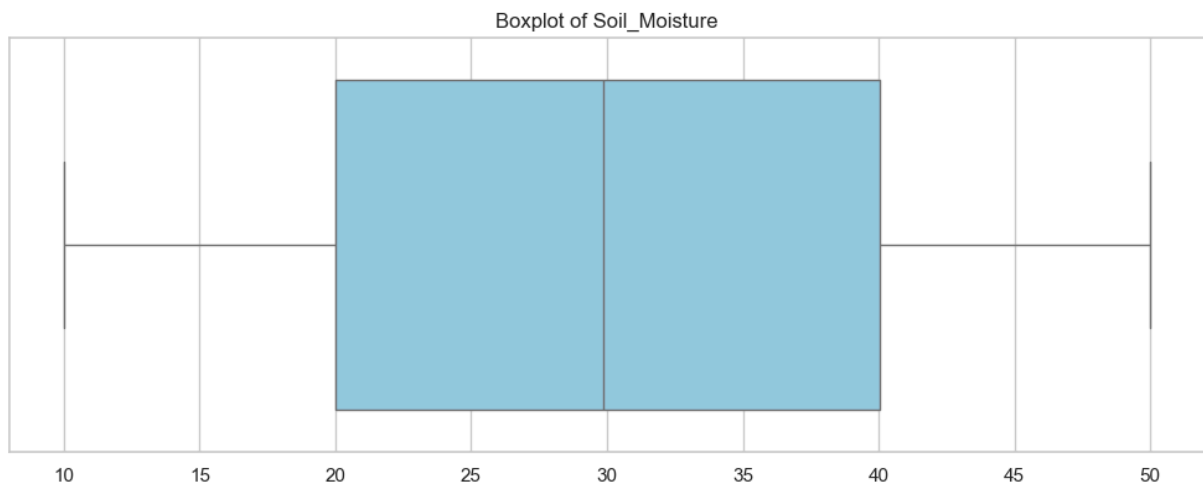
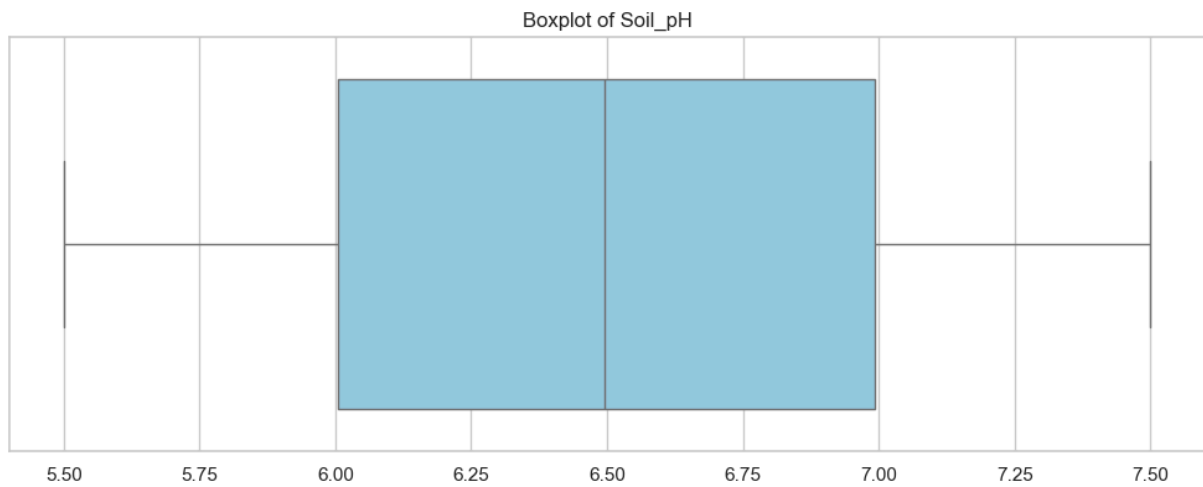
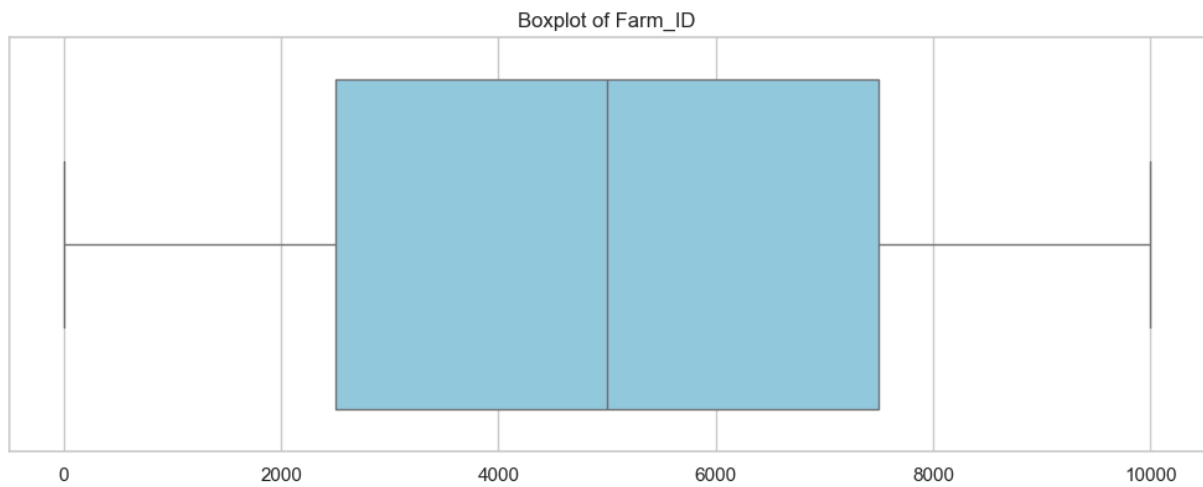
In [13]: # Set plot style
sns.set(style="whitegrid")

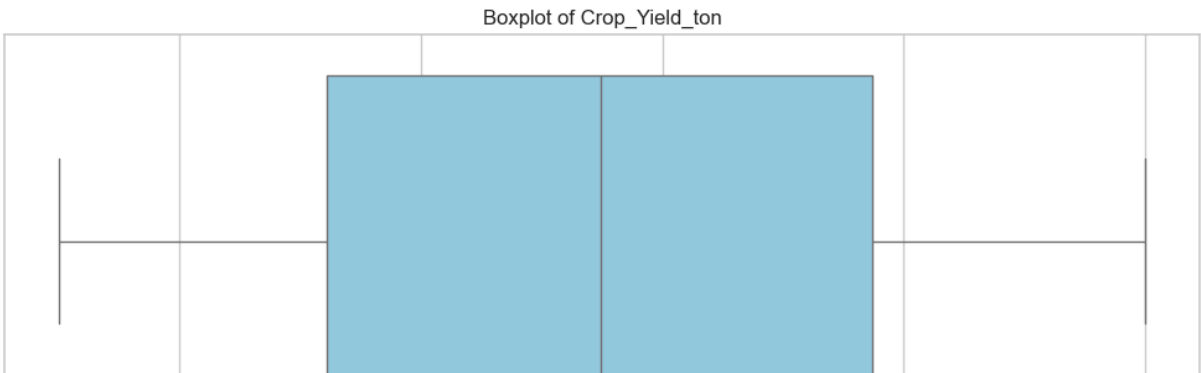
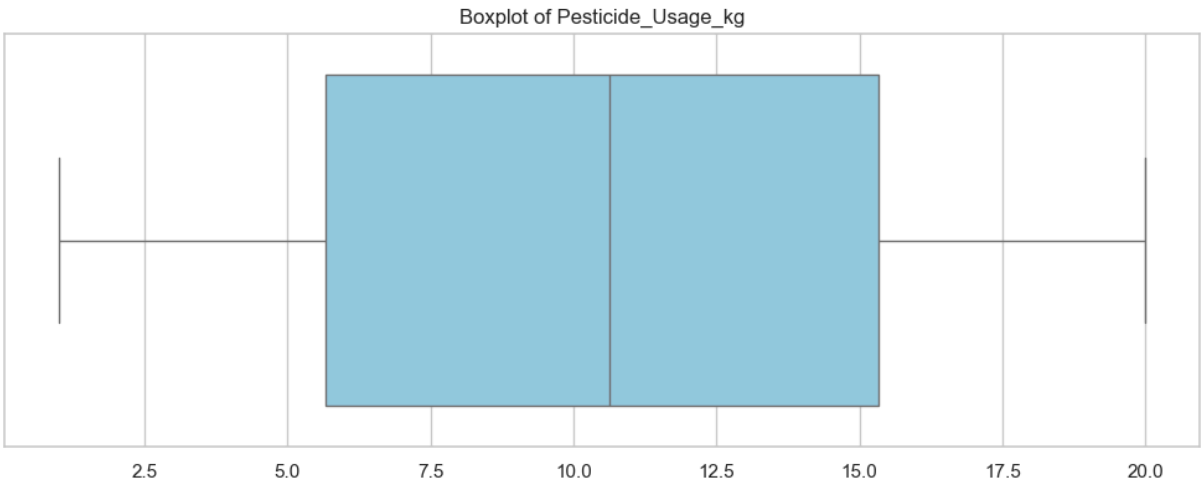
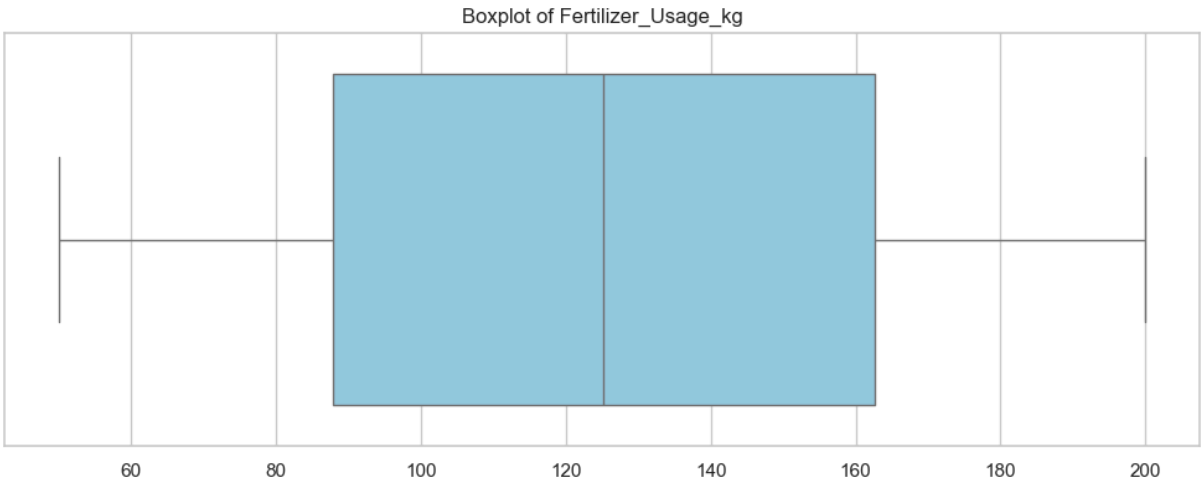
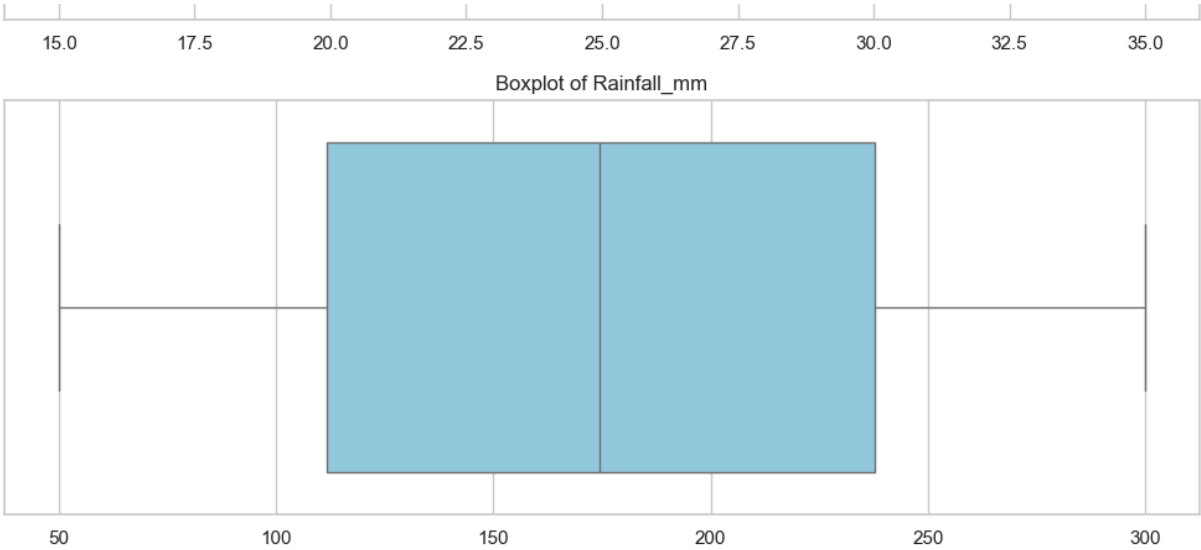
# Set figure size based on number of columns
plt.figure(figsize=(10, len(numerical_cols) * 4))

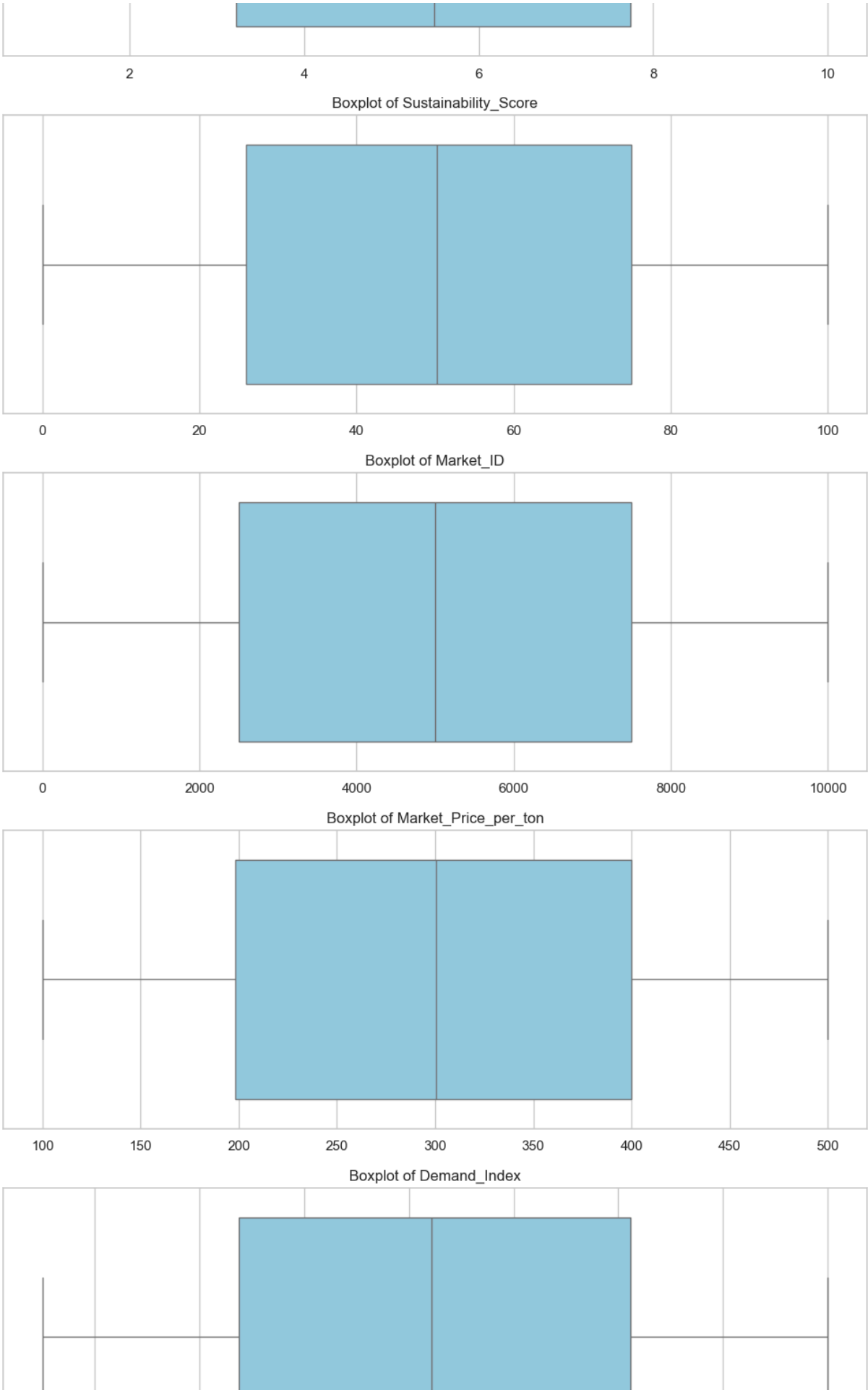
# Create a boxplot for each numerical column
for i, col in enumerate(numerical_cols, 1):
    plt.subplot(len(numerical_cols), 1, i)
    sns.boxplot(x=merged_data[col], color="skyblue")
    plt.title(f'Boxplot of {col}')
    plt.xlabel('')

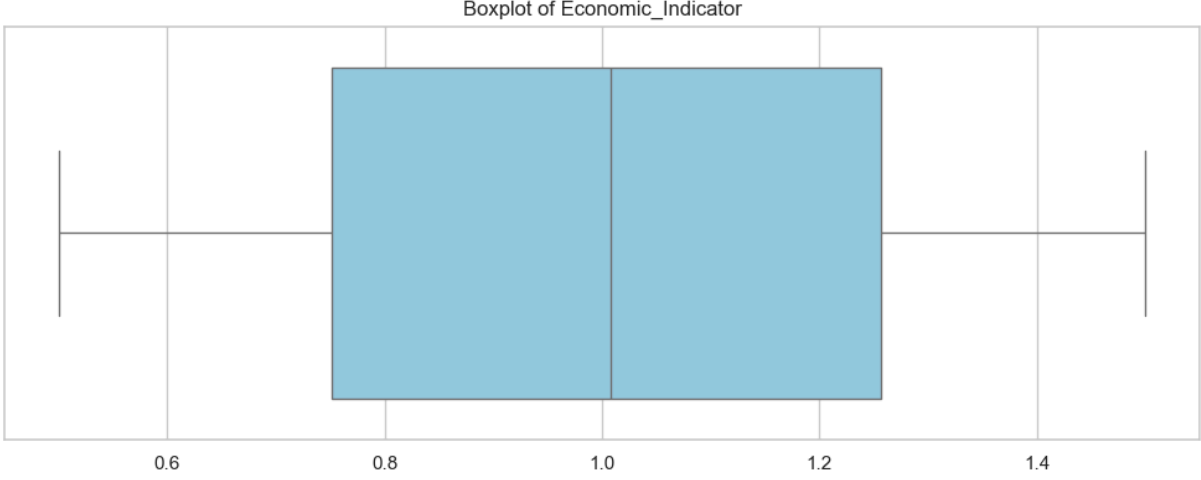
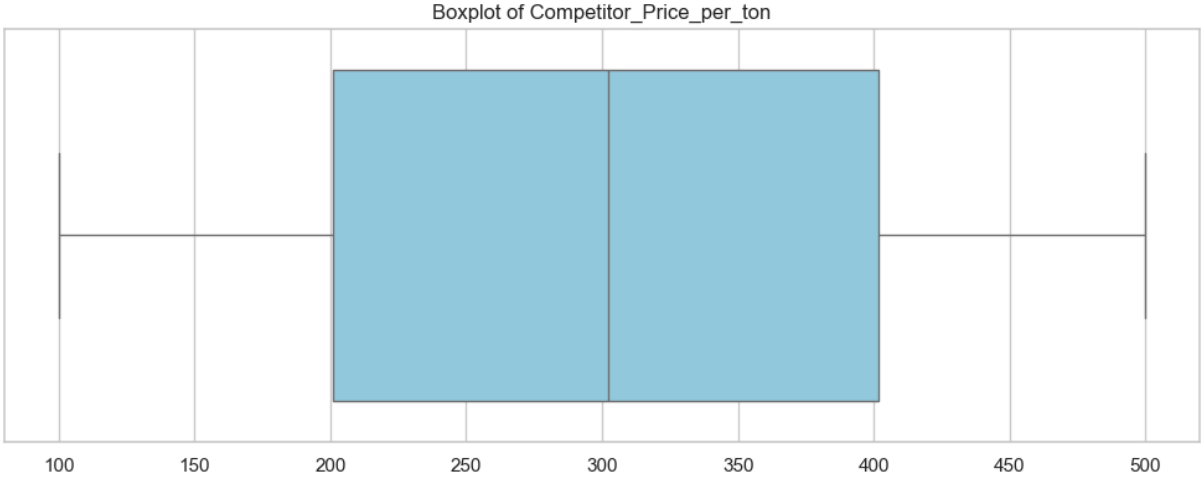
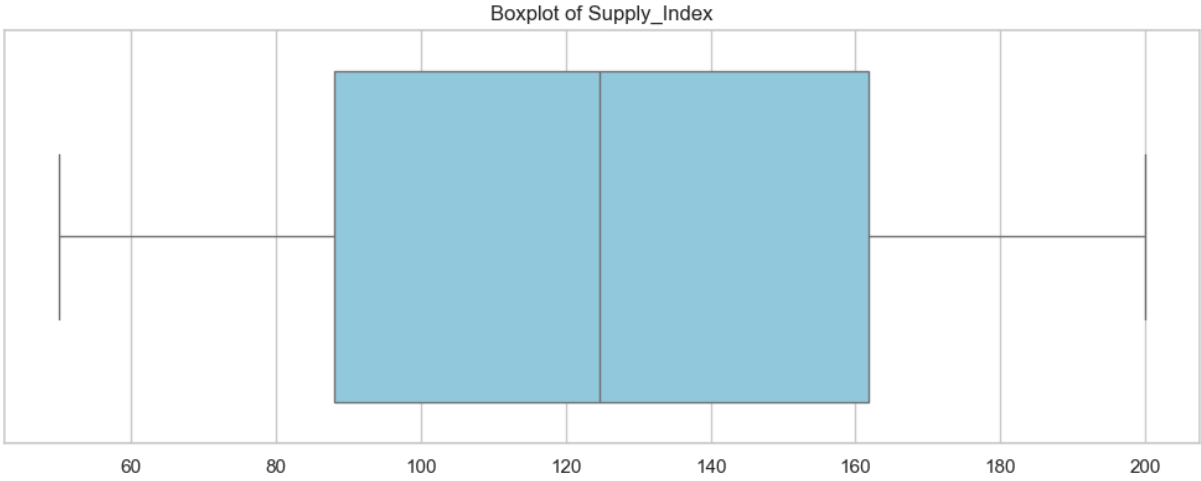
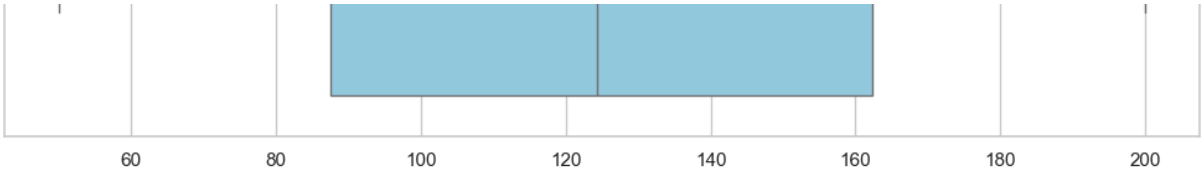
plt.tight_layout()
plt.show()

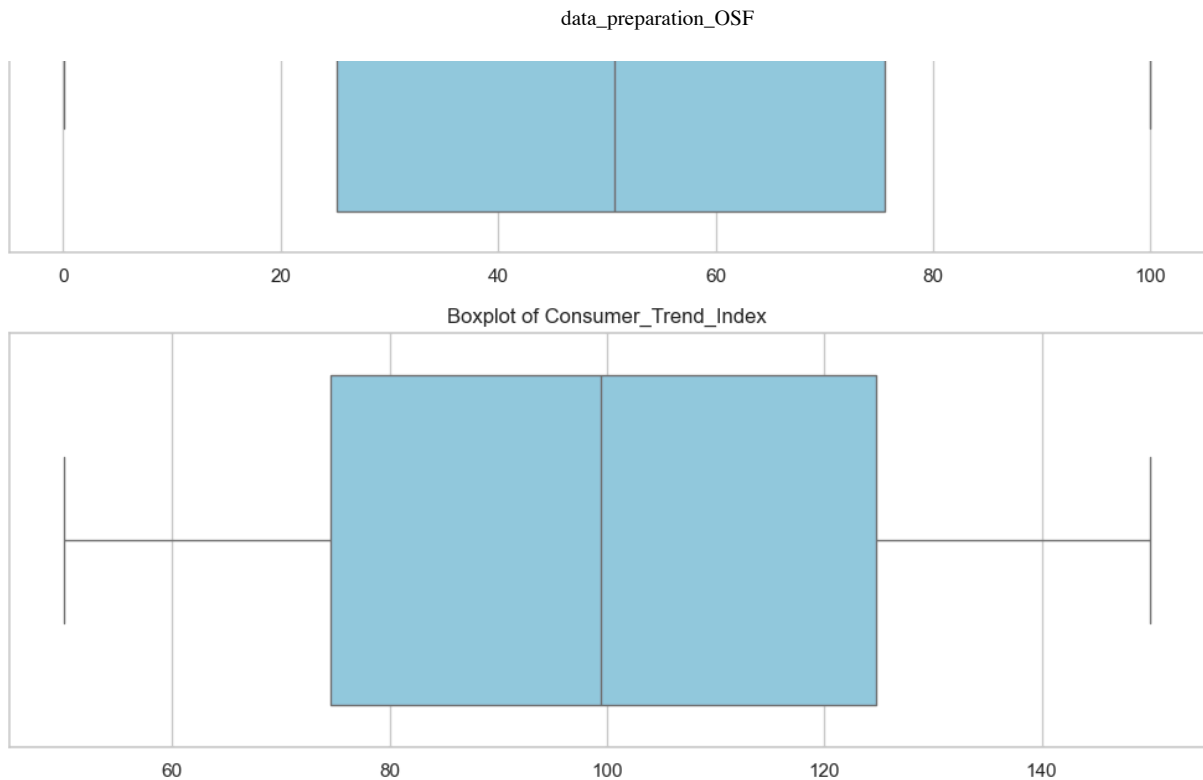
```











PLEASE, DO NOT PAY ATTENTION TO THE boxplot of Farm ID

3.3 Categorical Encoding

The merged dataset has two categorical columns:

- **Crop_Type** : (Wheat, Soybean, Corn, Rice) – requires one-hot encoding for modeling.
- **Seasonal_Factor** : (Low, Medium, High) – requires ordinal encoding due to its ordered nature.

We encode these to prepare for modeling.

```
In [14]: # One-hot encoding for Crop_Type
merged_data = pd.get_dummies(
    merged_data, columns=['Crop_Type'], prefix='Crop')
```

```
In [15]: # Ordinal encoding for Seasonal_Factor
seasonal_mapping = {'Low': 1, 'Medium': 2, 'High': 3}
merged_data['Seasonal_Factor'] = merged_data['Seasonal_Factor'].map(
    seasonal_mapping)
```

```
In [16]: print("\nColumns After Encoding:")
print(merged_data.dtypes)
```

Columns After Encoding:

Farm_ID	int64
Soil_pH	float64
Soil_Moisture	float64
Temperature_C	float64
Rainfall_mm	float64
Fertilizer_Usage_kg	float64
Pesticide_Usage_kg	float64
Crop_Yield_ton	float64
Sustainability_Score	float64
Market_ID	int64
Market_Price_per_ton	float64
Demand_Index	float64
Supply_Index	float64
Competitor_Price_per_ton	float64
Economic_Indicator	float64
Weather_Impact_Score	float64
Seasonal_Factor	int64
Consumer_Trend_Index	float64
Crop_Corn	bool
Crop_Rice	bool
Crop_Soybean	bool
Crop_Wheat	bool
dtype:	object

```
In [17]: # In this filtered dataframe visualization we can vefiry that our categorica
merged_data[['Seasonal_Factor', 'Crop_Corn', 'Crop_Rice',
              'Crop_Soybean', 'Crop_Wheat']]
```

Out[17]:

	Seasonal_Factor	Crop_Corn	Crop_Rice	Crop_Soybean	Crop_Wheat
0	1	False	False	False	True
1	1	False	False	False	True
2	3	False	False	False	True
3	1	False	False	False	True
4	3	False	False	False	True
...	...	...	...	...	...
25000325	1	False	False	False	True
25000326	1	False	False	False	True
25000327	1	False	False	False	True
25000328	3	False	False	False	True
25000329	3	False	False	False	True

25000330 rows x 5 columns

Observation: Crop_Type is split into binary columns (e.g., Crop_Wheat), and Seasonal_Factor is mapped to numerical values (1, 2, 3), making the dataset fully

numerical for modeling.

4. Feature Engineering

We derive five new features to enhance predictive modeling, focusing on resource efficiency, environmental impact, and interaction effects. Each feature is explained below.

4.1 Feature 1: Yield_per_Fertilizer

Logic: Measures efficiency of fertilizer use (tons of yield per kg of fertilizer). Higher values indicate better resource utilization, aligning with the goal of reducing resource use by 15%.

Line complete: `merged_data['Yield_per_Fertilizer'] = merged_data['Crop_Yield_ton'] / merged_data['Fertilizer_Usage_kg']`

```
In [18]: merged_data['Yield_per_Fertilizer'] = merged_data['Crop_Yield_ton'] / merged_data['Fertilizer_Usage_kg']
```

4.2 Feature 2: Resource_Intensity **Logic:** Combines fertilizer and pesticide usage (kg) per ton of yield to quantify total chemical input intensity. Lower values support sustainability goals.

Line complete: `merged_data['Resource_Intensity'] = (merged_data['Fertilizer_Usage_kg'] + merged_data['Pesticide_Usage_kg']) / merged_data['Crop_Yield_ton']`

```
In [19]: merged_data['Resource_Intensity'] = (merged_data['Fertilizer_Usage_kg'] + merged_data['Pesticide_Usage_kg']) / merged_data['Crop_Yield_ton']
```

4.3 Feature 3: Water_Stress_Index

Logic: Combines soil moisture and rainfall to assess water availability (normalized to 0-1). Low values indicate water stress, which may impact yield.

Line complete: `merged_data['Water_Stress_Index'] = 1 - ((merged_data['Soil_Moisture'] / 100) * (merged_data['Rainfall_mm'] / 300))`

```
In [20]: merged_data['Water_Stress_Index'] = 1 - ((merged_data['Soil_Moisture'] / 100) * (merged_data['Rainfall_mm'] / 300))
```

4.4 Feature 4: Soil_Weather_Interaction

Logic: Captures interaction between soil pH and temperature, as their combined effect may influence crop growth non-linearly.

Line complete: `merged_data['Soil_Weather_Interaction'] = merged_data['Soil_pH'] * merged_data['Temperature_C']`

```
In [21]: merged_data['Soil_Weather_Interaction'] = merged_data['Soil_pH'] * merged_data['Soil_Temperature']
```

4.5 Feature 5: Profit_Potential

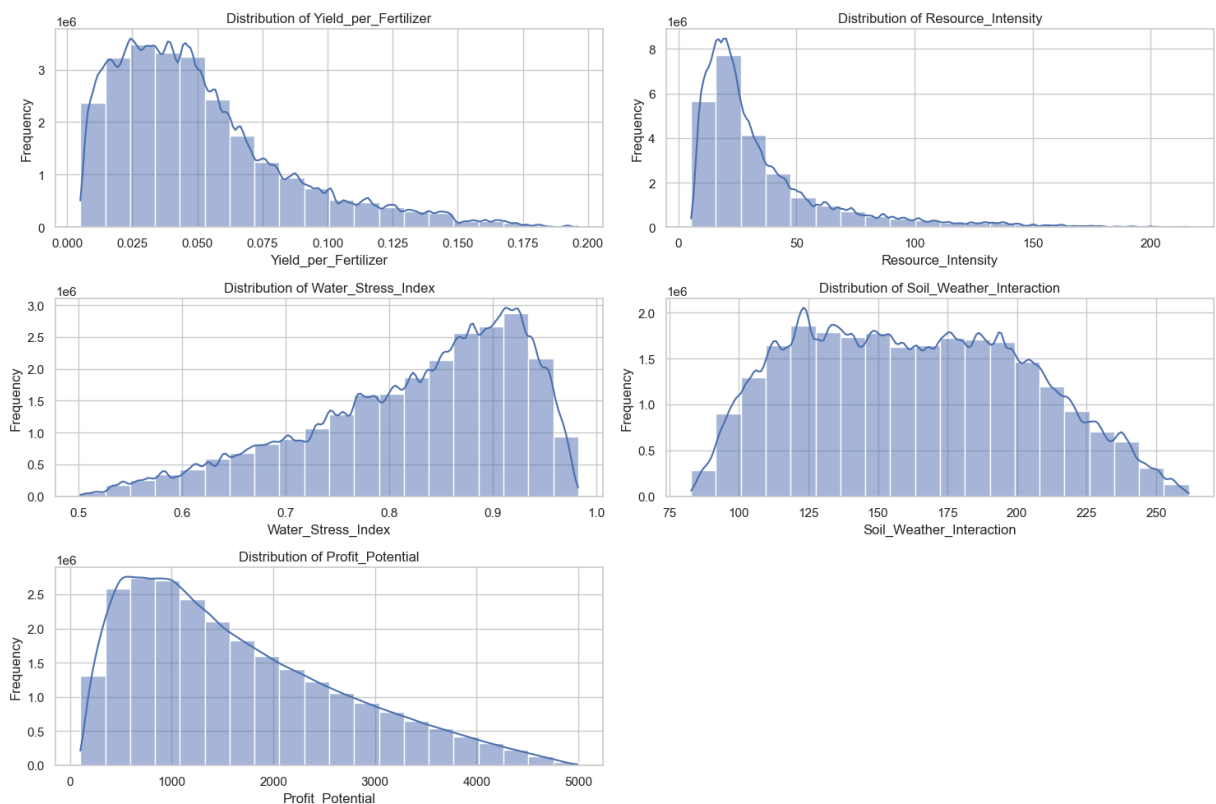
Logic: Estimates profit potential per ton of yield by multiplying yield by market price.

This integrates market dynamics into yield optimization.

Line complete: `merged_data['Profit_Potential'] = merged_data['Crop_Yield_ton'] * merged_data['Market_Price_per_ton']`

```
In [22]: merged_data['Profit_Potential'] = merged_data['Crop_Yield_ton'] * merged_data['Market_Price_per_ton']
```

```
In [23]: # Visualize new features
plt.figure(figsize=(15, 10))
for i, feature in enumerate(['Yield_per_Fertilizer',
                             'Resource_Intensity', 'Water_Stress_Index',
                             'Soil_Weather_Interaction', 'Profit_Potential'],
                             1):
    plt.subplot(3, 2, i)
    sns.histplot(merged_data[feature], bins=20, kde=True)
    plt.title(f'Distribution of {feature}')
    plt.xlabel(feature)
    plt.ylabel('Frequency')
plt.tight_layout()
plt.show()
```



Observations:

- Yield_per_Fertilizer:** Varies widely, indicating diverse fertilizer efficiency.

- **Resource_Intensity** : Skewed toward lower values, suggesting some farms use chemicals efficiently.
- **Water_Stress_Index** : Uniform, reflecting varied water availability.
- **Soil_Weather_Interaction** : Normal distribution, capturing combined soil-temperature effects.
- **Profit_Potential** : Right-skewed, driven by yield and price variability.

5. Conclusion

The Data Preparation phase successfully merged the primary and supplementary datasets on **Crop_Type** (by using a left join), creating a comprehensive dataset for modeling. No sampling was needed due to the manageable size and balanced crop distribution. The numerical columns were tested to know if they contain any outliers, by using the IQR method we encounter no outliers and it was verified by generating box plots for each numerical variable. Categorical variables (**Crop_Type** , **Seasonal_Factor**) were encoded using One-hot encoding for **Crop_Type** and Ordinal encoding for **Seasonal_Factor** to make easy the manipulation of the dataset for future CRISP-DM phases. Five new features were engineered to capture resource efficiency, environmental factors, and market dynamics, enhancing the dataset for predictive modeling. The dataset is now ready for the Modeling phase to achieve a 10% yield increase and 15% resource reduction.